

Technical Design Document

Nimble Sentences With Reinforcement Learning

Tyler Dinh, Lauren Gayle, Anling Zhu, Tenzin Dukdak,
Billy Zhao, Noah Wolk, Mark Seidel, Horlasy Demakpor

Software Team

{dinh.t, gayle.l, anling, dtenzin, billy.z,
n.j.wolk, m.m.seidel, horlasy.d}@wustl.edu

Elizabeth Joseph, Alexis Luna, Yuxuan Shen,
Kyle Okui, Gabe Minton

Integration Team

{elizabeth.j, alexis.l, notsure, k.okui, g.m.minton}@wustl.edu

August 29, 2026

Contents

1	Overview	4
2	Problem	5
2.1	The Core Problem	5
2.2	The Core Goal	5
3	Requirements	6
3.1	Out of Scope	6
3.2	Success Criteria	6
4	Architecture	7
4.1	VLA Dataset and Training	7
4.2	Fine-Tuned VLA Inference	8
4.3	Extension of Writing Range with a Linear Actuator	8
4.4	High-Level Overview (HLD)	9
4.5	API Design	9
4.6	Data Storage and Model	9
4.7	Application / Component Level Design (LLD)	9
5	Dependencies	10
5.1	Design Alternatives Considered	10
6	Cost Analysis	11
7	Failures and Risks	12
8	Non-Functional Requirements	13
8.1	Scalability	13
8.2	Latency and Availability	13
8.3	Maintainability	13
8.4	Security	13
9	Testing and Observability	14
9.1	Testing	14
9.2	Metrics and Alarms	14
10	Future Improvements	15
11	FAQs	16
A	Appendix A: Subtitle (Optional)	17
B	Glossary	18
C	References	19

Action Items

Software

1. LeRobot Dataset Collection

Collect a `LeRobotDataset` for eventual VLA fine-tuning. This is the highest priority task.

- Please follow the notes in the Architecture section about dataset collection.
- Prior to collection, the environment must be configured, including arm attachments (Logitech camera), and environmental objects (whiteboard, marker holder, etc.).

2. VLA Fine-Tuning

Fine-tune NORA 1.5 for handwriting tasks. Zero-shot inference of the NORA 1.5 VLA has been observed to not perform well in this domain, so we will fine-tune it.

- NORA 1.5's pretraining data likely contains no robotic writing examples, making fine-tuning essential for this specific scenario of writing.
- Fine-tuning will use a HuggingFace `LeRobotDataset` via the fine-tuning script provided in the NORA 1.5 GitHub repository.
- The fine-tuned VLA will be released publicly on HuggingFace. This work has potential for academic writing, like a technical white paper or report given the project's structure. The fine-tuned model could also be shared on social media to gauge interest from researchers and practitioners. A faculty collaborator may be worth pursuing in the future after we have completed this project.

Integration

1. Arm and Camera Wiring

Physical wiring of the arm and camera into the compute environment must be established and discussed with the team. Since the Roboserver will be running VLA inference and all associated scripts, a reliable communication channel between the arm, camera, and Roboserver is required.

2. Physical Environment Setup

The physical environment of the arm must be configured for data collection and fine-tuning as closely as possible. Mismatches between virtual and physical environments are a primary source of performance degradation.

3. Linear Actuator Integration (*in progress*)

A linear actuator enables the arm to traverse the writing surface horizontally, extending its effective writing range. Without this, the robot cannot produce arbitrarily long text on a surface larger than its static range of motion. There will need to be discussion of how communication will travel both ways through the RoboServer.

1 Overview

The purpose of this document is to present the technical design of a robotic arm that can perform handwriting tasks in a physical environment for basic sentences using fine-tuning via reinforcement learning (RL) on a Vision-Language Action model (VLA).

This document is intended for a technical audience with a background in computer science, robotics, or machine learning. Non-technical readers may find high-level summaries useful but should refer to supplementary documentation at the end of this document for background on the underlying mechanisms employed.

2 Problem

Current robot arms used for writing tasks operate with **static, pre-programmed motions**. Each specific writing task requires its own custom motion program, which has several critical limitations:

2.1 The Core Problem

If a robot needs to write something new, engineers must:

- Manually program the exact trajectory for each letter or word.
- Hard-code the sequence of movements.
- Test, refine, and repeat this entire process for every new writing task.

An autonomous robot that generalizes its writing capabilities would:

- Scale efficiently: Write any arbitrary text without reprogramming.
- Adapt to variations: Handle different sizes and styles of text.
- Learn from examples: Improve through demonstration rather than explicit programming.

2.2 The Core Goal

The goal is to develop a robot writing system that can:

- Accept arbitrary text input (like “Hello World” or even more arbitrary, like “Write me a haiku.”)
- Dynamically generate appropriate writing motions.
- Execute smooth, legible handwriting without pre-programmed trajectories for each specific string.

3 Requirements

TODO! Describe all requirements imposed by the problem. Write from the end-user's perspective. Include user stories or use cases.

Example Use Cases

- As a x, I want to y.
- As a x, I want y.

3.1 Out of Scope

TODO! List what is explicitly out of scope to avoid misunderstandings.

3.2 Success Criteria

TODO! Describe how success will be measured once the solution is in production. Include measurable metrics such as performance, scalability, or cost efficiency.

Example Metrics

- Time-to-market reduced by 90%.
- Supports 10,000 users with 100ms P90 latency.
- It can write sentences in English legibly with X% accuracy.

4 Architecture

Describe the architecture of the proposed solution using text, diagrams, and bullet points. Explain why this design was chosen.

25-26 Architecture

VLA: NORA 1.5 Data Collection Framework: LeRobot Arm: WidowX AI Trossen Base CV: OpenCV + Logitech Additional VLM/LLM for Data Preprocessing: Local Qwen3-4B instance, Programming Languages: Mainly Python, JSON for data. Compute: RoboServer

4.1 VLA Dataset and Training

During operation, the VLA will be asked by the user to write a sentence or word in the format of Write 'THING_TO_WRITE'.

Q: How do we ensure the VLA can understand all text inputs? We need to collect lots of diverse data.

Q: How do we collect diverse data for the text of a sentence? We need to learn how to write characters in that language so that text can be broken down into atomic units.

For collecting diverse data points for writing English text, we define a control script to write characters given their stroke-by-stroke physical definitions. Each data point for a character contains:

1. The writing instruction: Write 'character'
2. The vision: camera input captured before writing that stroke
3. The stroke movement: the arm trajectory required to draw the next stroke of the desired character

Q: Why not just train on words? VLAs have sparse rewards, only receiving a reward signal at the end of the sequence. Writing a word takes many timesteps. More fundamentally, the VLA outputs a single 6-DOF action vector per inference pass—there is no single action that writes a word. Writing even a short word requires hundreds of sequential arm poses, plus pen-lifts within characters (e.g., the dot in 'i'), between characters, and between words. Training at the character level avoids all of these issues.

Control Flow

We can write each character given their stroke-by-stroke physical definitions, and collect their datapoints (for each stroke of a given character) through the Trossen Data Collection framework.

```
"Write 'A'"
-> parse -> ['A']
-> for each character:
    -> look up stroke count, physical motions in defined constant
    -> for each stroke in range(stroke_count):
        -> start recording an 'episode' for data collection
        -> collect current frame that the robot sees
        -> draw stroke using trossen library
        -> collect the 6-DOF tuple / action that is used to draw the stroke
```

```

-> the data point is the writing instruction, frame, action (stroke)
-> scripted pen-lift + reposition to next character start position
-> done

```

4.2 Fine-Tuned VLA Inference

A sentence is decomposed hierarchically:

- A **sentence** decomposes into words, separated by scripted spaces.
- A **word** decomposes into characters, separated by scripted smaller spaces.
- A **character** decomposes into strokes. A **stroke** is the atomic unit of VLA inference.

For example, ‘A’ has 3 strokes and ‘I’ has 1 stroke. The VLA performs one inference pass per stroke, and the stroke count per character is physically defined in code, so the system always knows exactly how many inference passes to run. Each inference pass receives:

1. The writing instruction for the *character*: Write ‘character’
2. The vision: camera input captured before executing that stroke
3. Output: the stroke movement vector used to actuate the arm

Pen-lifts and repositioning between characters and words are **scripted**, not learned, so they do not require VLA inference.

Control Flow

```

"Write 'hello'"
-> parse -> ['h', 'e', 'l', 'l', 'o']
-> for each character:
  -> look up stroke count in physically defined constant
  -> run VLA inference stroke_count times WITH fine-tuned model
  -> scripted pen-lift + reposition to next character start position
-> done

"Write 'hello world'"
-> parse -> [['h','e','l','l','o'], ['w','o','r','l','d']]
-> for each word:
  -> for each character:
    -> look up stroke count in physically defined constant
    -> run VLA inference stroke_count times WITH fine-tuned model
    -> scripted pen-lift + reposition to next character start position
  -> scripted larger pen-lift and reposition for word gap
-> done

```

4.3 Extension of Writing Range with a Linear Actuator

Writing text is difficult, as the phrase ‘hello world this is a cool sentence’ with the WidowX arm can only be written in the range of action given by the robot and its camera. A linear actuator provides a ‘sliding window’ over the writing surface, enabling arbitrarily long text output.

Text Planning

Before any motion occurs, the full spatial layout of the text should be pre-computed:

- Determine character width and inter-character spacing in physical units (cm).
- Map the full string to an absolute coordinate sequence along the writing axis.
- Determine the total horizontal distance required.

Text Chunking by Reachability

Given the current position of the arm, a **range of action** is defined, which is the zone the writing tool can write in at the current position. The input text to the start of the program is greedily chunked, meaning that characters are grouped until the next character would fall outside the window. Chunk boundaries are placed *between characters*, but should never be chunked mid-character.

Slide by Inference on a Chunk

For each chunk, the VLA (NORA 1.5) executes the writing motions normally. When the chunk is complete, the actuator must activate, moving X distance based on where the next chunk is. Actuator movement is intentionally separate from VLA inference, as it is purely physical and does not need to be learned. After sliding, a brief re-anchoring step determines where the last written character ended to determine where the arm should resume movement. The VLA is then re-prompted with the next chunk of text.

4.4 High-Level Overview (HLD)

List all logical system components:

- comp1
- comp2
- comp3
- comp4
- comp5

Include diagrams as needed:

4.5 API Design

List all APIs used for interaction between users or services. Include HTTP methods, payloads, versions, and example requests/responses. Discuss future evolution of the APIs.

4.6 Data Storage and Model

Describe the data model and database choice. Estimate data volume, forecast growth, and justify scalability. Discuss data pipelines, ingestion, and pre-processing if applicable.

4.7 Application / Component Level Design (LLD)

Provide detailed design of each system component, including data flow and control flow diagrams.

USE DRAW.IO for this (can install on your computer)

5 Dependencies

List external and internal dependencies. Document assumptions and risks associated with these dependencies.

5.1 Design Alternatives Considered

Discuss alternative designs evaluated. Provide a comparison table highlighting trade-offs.

Option	Pros	Cons
Option A	Scalable, cost-efficient	Complex to maintain
Option B	Easy to implement	Limited scalability

6 Cost Analysis

Estimate infrastructure and operational costs. Plan for future growth and scalability.

7 Failures and Risks

Discuss potential system failures such as:

- Dependency failures
- Traffic overflow
- Performance degradation
- Logic bugs

Identify risks such as external dependencies or resource limitations. Include potential mitigation strategies.

8 Non-Functional Requirements

Cover scalability, availability, maintainability, reliability, latency, and security.

8.1 Scalability

Expected users, transactions, and data volume.

8.2 Latency and Availability

Define SLAs (e.g., P99, P50 latency targets).

8.3 Maintainability

Explain maintenance expectations and ownership.

8.4 Security

Describe security measures and required compliance levels.

9 Testing and Observability

Outline strategies for testing, monitoring, and alerting.

9.1 Testing

Explain testing approach: unit, integration, A/B, or stress tests.

9.2 Metrics and Alarms

List key performance metrics, dashboards, and alert configurations.

10 Future Improvements

List planned features or improvements for future releases.

11 FAQs

Include frequently asked questions and their answers.
question? answer here

A Appendix A: Subtitle (Optional)

Include detailed technical analysis or supporting data.

B Glossary

ex1 example definition

C References

- Service 1: <https://example.com>